# Модуль 4

## HTTP-сервер на чистом Node.js

*http модуль • Запросы и ответы • Роутинг • Статические файлы • JSON API • Query/Body*

## 4.1 Как работает HTTP

Прежде чем писать сервер, важно понять протокол HTTP. Когда вы открываете сайт в браузере, происходит следующее:

- Браузер (клиент) отправляет HTTP-запрос на сервер: метод (GET, POST...), URL, заголовки, тело
- Сервер получает запрос, обрабатывает его и отправляет HTTP-ответ: статус-код (200, 404...), заголовки, тело
- Браузер получает ответ и отображает его

Основные HTTP-методы:

- GET — получить данные (страницу, список пользователей)
- POST — создать новый ресурс (зарегистрировать пользователя, создать пост)
- PUT — полностью заменить ресурс
- PATCH — частично обновить ресурс
- DELETE — удалить ресурс

Основные статус-коды:

- 200 OK — запрос успешен
- 201 Created — ресурс создан
- 204 No Content — успех, но тело ответа пустое (после DELETE)
- 400 Bad Request — ошибка в запросе клиента
- 401 Unauthorized — требуется аутентификация
- 403 Forbidden — доступ запрещён
- 404 Not Found — ресурс не найден
- 500 Internal Server Error — ошибка на сервере

## 4.2 Простой HTTP-сервер

Node.js имеет встроенный модуль http, который позволяет создать HTTP-сервер без установки дополнительных пакетов.

Функция createServer принимает callback, который вызывается при каждом входящем запросе:

```
import { createServer } from 'http';


const server = createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end('<h1>Привет, Node.js!</h1>');
});


server.listen(3000, () => {
    console.log('Сервер запущен: http://localhost:3000');
});
```

Разберём этот код:

- createServer(callback) — создаёт сервер. Callback получает два объекта: req (запрос) и res (ответ)
- req (IncomingMessage) — содержит информацию о запросе: req.method, req.url, req.headers
- res (ServerResponse) — используется для отправки ответа клиенту
- res.writeHead(statusCode, headers) — устанавливает статус-код и заголовки
- res.end(body) — отправляет тело ответа и завершает его. Каждый запрос ОБЯЗАТЕЛЬНО должен завершаться вызовом res.end()
- server.listen(port) — запускает сервер на указанном порту

Заголовок Content-Type сообщает браузеру, какой тип данных мы отправляем: text/html для HTML, application/json для JSON, text/css для CSS и т.д. Без charset=utf-8 кириллица может отображаться некорректно.

Давайте создадим новую папку и перейдем в нее

```
mkdir test-http-server
cd test-http-server
```

Инициализируем проект с помощью npm init -y и использованием модулей ESM (помните “type”: “module” добавить в “package.json” )

Далее создадим новый файл index.js с содержимым

```
import { createServer } from 'http';


const server = createServer((req, res) => {
    res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
    res.end('<h1>Привет, Node.js!</h1>');
});


server.listen(3000, () => {
    console.log('Сервер запущен: http://localhost:3000');
});
```

И запустим веб-сервер, выполнив

```
node index.js
```

Теперь в браузере откройте http://localhost:3000 и увидите надпись

**Привет, Node.js!**

Это значит, что ваш скрипт инициализировал веб-сервер и запустился.

Теперь можно остановить веб-сервер, нажав CTRL + C

## 4.3 Роутинг

Роутинг — это определение того, какой код выполнять в зависимости от URL и HTTP-метода запроса. На чистом Node.js роутинг реализуется через условия if/else.

### Пошаговый пример: роутинг GET/POST и чтение body

Сделаем API с несколькими роутами и проверим корректную обработку JSON-тела POST-запроса.

Обновите index.js:

```
import { createServer } from 'http';


const users = [
    { id: 1, name: 'Алексей' },
    { id: 2, name: 'Мария' },
];


const server = createServer((req, res) => {
    const { method, url } = req;
    res.setHeader('Content-Type', 'application/json; charset=utf-8');
```

```
    if (method === 'GET' && url === '/') {
        res.writeHead(200);
        res.end(JSON.stringify({ message: 'Главная страница' }));
    } else if (method === 'GET' && url === '/api/users') {
        res.writeHead(200);
        res.end(JSON.stringify(users));
    } else if (method === 'POST' && url === '/api/users') {
        let body = '';
        req.on('data', (chunk) => body += chunk);
        req.on('end', () => {
            try {
                const user = JSON.parse(body);
                user.id = Date.now();
                users.push(user);
                res.writeHead(201);
                res.end(JSON.stringify(user));
            } catch {
                res.writeHead(400);
                res.end(JSON.stringify({ error: 'Некорректный JSON'
}));
            }
        });
    } else {
        res.writeHead(404);
        res.end(JSON.stringify({ error: 'Не найдено' }));
    }
});

server.listen(3000, () => {
    console.log('Сервер запущен: http://localhost:3000');
});
```

Запустите сервер:

```
node index.js
```

Проверьте роуты с помощью консольной утилиты curl. Запустите по очереди следующие команды

```
curl -i http://localhost:3000/

curl -i http://localhost:3000/api/users

curl -i -X POST http://localhost:3000/api/users -H "Content-Type:
application/json" -d '{"name":"Oleg"}'
```

```
curl -i -X POST http://localhost:3000/api/users -H "Content-Type:
application/json" -d "not-json"
```

Что проверить в результате:

- GET / возвращает 200 и JSON с message
- GET /api/users возвращает массив пользователей
- POST /api/users с валидным JSON возвращает 201 и созданного пользователя
- POST /api/users с невалидным JSON возвращает 400

Роуты, которые обрабатывают GET запросы, например / или /api/users можно запустить прямо из браузера, открыв ссылки
http://localhost:3000/
и
http://localhost:3000/api/users

## 4.4 Query-параметры

Query-параметры — это данные, передаваемые в URL после знака вопроса: /api/search?q=node&page=2. Для их парсинга используется встроенный класс URL:

```
const server = createServer((req, res) => {
    const parsedUrl = new URL(req.url, `http://${req.headers.host}`);
    const pathname = parsedUrl.pathname;  // /api/search
    const query = Object.fromEntries(parsedUrl.searchParams);
    // query = { q: 'node', page: '2' }

    if (pathname === '/api/search') {
        res.writeHead(200, { 'Content-Type': 'application/json' });
        res.end(JSON.stringify({ query, results: [] }));
    }
});
```

*Важно*: все значения query-параметров — строки. Если вам нужно число, преобразуйте явно:

const page = parseInt(query.page) или +query.page.

**Пошаговый пример: query-параметры (фильтрация и пагинация)**

Добавим endpoint с фильтрацией по автору и простой пагинацией через page/limit.

Для добавления нового роута GET /api/messages обновите файл следующим образом:

```
import { createServer } from 'http';

const users = [
    { id: 1, name: 'Алексей' },
    { id: 2, name: 'Мария' },
];

const messages = [
    { id: 1, author: 'Alex', text: 'Hello' },
    { id: 2, author: 'Maria', text: 'Hi' },
    { id: 3, author: 'Alex', text: 'Node.js' },
];

const server = createServer((req, res) => {
    const { method, url } = req;
    res.setHeader('Content-Type', 'application/json; charset=utf-8');

    if (method === 'GET' && url === '/') {
        res.writeHead(200);
        res.end(JSON.stringify({ message: 'Главная страница' }));
    } else if (method === 'GET' && url === '/api/users') {
        res.writeHead(200);
        res.end(JSON.stringify(users));
    } else if (method === 'POST' && url === '/api/users') {
        let body = '';
        req.on('data', (chunk) => body += chunk);
        req.on('end', () => {
          try {
            const user = JSON.parse(body);
            user.id = Date.now();
            users.push(user);
            res.writeHead(201);
            res.end(JSON.stringify(user));
          } catch {
            res.writeHead(400);
            res.end(JSON.stringify({ error: 'Некорректный JSON' }));
          }
        });
    } else if (method === "GET" && url.startsWith("/api/messages")) {
      const parsedUrl = new URL(url, `http://${req.headers.host}`);
      const query = Object.fromEntries(parsedUrl.searchParams);
```

```
      const page = Number(query.page ?? 1);
      const limit = Number(query.limit ?? 10);

      let filtered = messages;
      if (query.author) {
        filtered = filtered.filter((m) => m.author === query.author);
      }

      const start = (page - 1) * limit;
      const paged = filtered.slice(start, start + limit);

      res.writeHead(200);
      res.end(JSON.stringify({ query, total: filtered.length, data:
paged }));
    } else {
      res.writeHead(404);
      res.end(JSON.stringify({ error: 'Не найдено' }));
    }
});

server.listen(3000, () => {
    console.log('Сервер запущен: http://localhost:3000');
});
```

Проверьте запросы:

```
curl -i "http://localhost:3000/api/messages"
curl -i "http://localhost:3000/api/messages?author=Alex"
curl -i "http://localhost:3000/api/messages?author=Alex&page=1&limit=1"
```

Что важно помнить:

- Все query-параметры приходят строками
- page и limit нужно явно приводить к числам
- Фильтры можно безопасно комбинировать: author + page + limit

## 4.5 Отдача статических файлов

Статические файлы — это HTML, CSS, JavaScript, изображения, которые отдаются клиенту без обработки. Сервер просто читает файл с диска и отправляет его. Для правильной работы нужно определить MIME-тип файла по его расширению:

```
import { createServer } from 'http';
import { readFile } from 'fs/promises';
```

```
import { join, extname } from 'path';

// Таблица MIME-типов
const MIME = {
    '.html': 'text/html',
    '.css':  'text/css',
    '.js':   'text/javascript',
    '.json': 'application/json',
    '.png':  'image/png',
    '.jpg':  'image/jpeg',
    '.svg':  'image/svg+xml',
    '.ico':  'image/x-icon',
};

const server = createServer(async (req, res) => {
    // Определяем путь к файлу
    let filePath = join('public', req.url === '/' ? 'index.html' :
req.url);
    const ext = extname(filePath);
    const contentType = MIME[ext] || 'application/octet-stream';

    try {
        const data = await readFile(filePath);
        res.writeHead(200, { 'Content-Type': contentType });
        res.end(data);
    } catch {
        res.writeHead(404, { 'Content-Type': 'text/html; charset=utf-8'
});
        res.end('<h1>404 — Файл не найден</h1>');
    }
});

server.listen(3000);
```

Этот сервер ищет запрошенные файлы в папке public/. Запрос GET / отдаст public/index.html, запрос GET /css/style.css отдаст public/css/style.css. Если файл не найден — вернёт 404.

### Пошаговый пример: статические файлы и 404

Соберём минимальный static-сервер с выдачей index.html, style.css и корректным ответом 404.

Создайте новую папку проекта и перейдите в нее затем инициализируйте новый проект npm init -y

```
mkdir test-static-server
cd test-static-server
```

Создайте папку public и css внутри:

```
mkdir -p public/css
```

Создайте файл public/index.html

```
<!doctype html>

<html><head><meta charset="utf-8"><link rel="stylesheet"
href="/css/style.css"></head>

<body><h1>Static OK</h1></body></html>
```

И файл public/css/style.css

```
body { font-family: sans-serif; color: #1f2937; }
```

Создайте index.js:

```
import { createServer } from 'http';
import { readFile } from 'fs/promises';
import { join, extname } from 'path';


const MIME = {
    '.html': 'text/html; charset=utf-8',
    '.css': 'text/css; charset=utf-8',
    '.js': 'text/javascript; charset=utf-8',
    '.json': 'application/json; charset=utf-8',
};


const server = createServer(async (req, res) => {
    const rawPath = req.url === "/" ? "/index.html" : req.url;
    const filePath = join("public", rawPath);
    const ext = extname(filePath);
    const contentType = MIME[ext] ?? "application/octet-stream";


    try {
        const data = await readFile(filePath);
        res.writeHead(200, { "Content-Type": contentType });
        res.end(data);
    } catch {
        res.writeHead(404, { "Content-Type": "text/html; charset=utf-8"
});
        res.end("<h1>404 — Файл не найден</h1>");
    }
});


server.listen(3000);
```

Шаг 4. Запустите сервер и проверьте:

```
node index.js
curl -i http://localhost:3000/
curl -i http://localhost:3000/css/style.css
curl -i http://localhost:3000/not-found
```

Что проверить в результате:

- / отдаёт HTML-файл
- /css/style.css отдаёт CSS с правильным Content-Type
- Несуществующий путь возвращает 404

## Практика

**Задание 4.1:**

- Создайте HTTP-сервер с роутами: GET / (HTML-страница), GET /about, GET /api/time (текущее время в JSON)
- Добавьте POST /api/messages — принимает JSON { text, author }, сохраняет в массив с id и timestamp
- Добавьте GET /api/messages — возвращает все сообщения
- Реализуйте отдачу статических файлов из папки public/ (создайте index.html и style.css)
- Добавьте обработку query-параметров: GET /api/messages?author=Alex фильтрует по автору
- Протестируйте POST-запросы через curl или Postman

**Задание 4.2** (вопросы для самопроверки):

- Какая роль у объектов req и res в createServer callback?
- Почему каждый HTTP-запрос должен завершаться вызовом res.end()?
- Чем отличаются статус-коды 200, 201, 204 и когда каждый уместен?
- Как правильно обработать 404 в роутинге на чистом Node.js?

- Почему тело POST-запроса читается через события data/end, а не доступно сразу?
- Какие риски возникают при JSON.parse(body) и как их обработать безопасно?
- Что даёт установка Content-Type в ответе и к чему приводит ошибка в MIME-типе?
- Как извлечь query-параметры через URL и почему page/limit нужно приводить к Number?
- Как реализовать фильтрацию GET /api/messages?author=Alex на уровне сервера?
- Чем отличается отдача JSON API и статических файлов в одном HTTP-сервере?
- Почему для static-файлов важно определять расширение и выставлять MIME?
- Какие базовые проверки нужно добавить, чтобы безопасно отдавать файлы из public/?
- Какими командами curl проверить GET, POST и 404 сценарии для вашего сервера?
- В каких местах сервер на чистом Node.js начинает “болеть” и почему затем используют Express?
- Как бы вы организовали код, если роутов станет 30+: один файл, модули, таблица роутов?